

Unit II: Managing Tables and Data in SQL (MySQL)

Detailed Notes with Related Queries and Examples

This unit teaches **how to create database tables, modify them, store data, retrieve data, and manipulate data using SQL commands.**

1. Creating Tables (DDL - Data Definition Language)

A **table** is a collection of rows and columns used to store data.

Syntax

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints,  
    ...  
);
```

Example

Suppose we are developing a **College Database**.

Step 1: Create Department Table

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50) NOT NULL  
);
```

Output

Dept_ID	Dept_Name
---------	-----------

No data yet

Step 2: Create Student Table

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,
```

```
Name VARCHAR(50) NOT NULL,  
Age INT,  
Gender CHAR(1),  
City VARCHAR(30),  
Marks DECIMAL(5,2),  
Dept_ID INT,  
FOREIGN KEY (Dept_ID)  
REFERENCES Department(Dept_ID)  
);
```

Here

Student table depends upon Department table because of **Foreign Key**.

Relationship

Department

Dept_ID (PK)

|

|

↓

Student

Dept_ID (FK)

2. Constraints

Constraints maintain data accuracy.

A. PRIMARY KEY

Uniquely identifies each record.

Cannot be NULL.

Example

Student_ID INT PRIMARY KEY

Example Data

Student_ID	Name
------------	------

101	Rahul
-----	-------

102	Aman
-----	------

Wrong

INSERT INTO Student

VALUES (101,'Priya',20,'F','Delhi',90,1);

Error

Duplicate Entry

B. NOT NULL

Column cannot contain NULL values.

Name VARCHAR(50) NOT NULL

Wrong

INSERT INTO Student

VALUES

(103,NULL,20,'M','Delhi',89,1);

Error

Column Name cannot be NULL

C. UNIQUE

No duplicate values allowed.

Email VARCHAR(100) UNIQUE

D. CHECK

Checks condition.

Age INT CHECK(Age>=18)

Wrong

Age = 15

Rejected.

E. DEFAULT

Automatically inserts value.

City VARCHAR(30) DEFAULT 'Bhopal'

If City not entered

City = Bhopal

F. FOREIGN KEY

Maintains relationship between tables.

FOREIGN KEY(Dept_ID)

REFERENCES Department(Dept_ID)

Cannot insert department that does not exist.

Wrong

Dept_ID = 50

if Department 50 is absent.

3. ALTER TABLE

Used to modify existing table.

Add Column

ALTER TABLE Student

```
ADD Phone VARCHAR(15);
```

Before

```
|ID|Name|
```

After

```
|ID|Name|Phone|
```

Modify Column

```
ALTER TABLE Student
```

```
MODIFY Phone VARCHAR(20);
```

Rename Column

```
ALTER TABLE Student
```

```
RENAME COLUMN Phone TO Mobile;
```

Drop Column

```
ALTER TABLE Student
```

```
DROP COLUMN Mobile;
```

Add Constraint

```
ALTER TABLE Student
```

```
ADD CONSTRAINT chk_marks
```

```
CHECK(Marks>=0);
```

Drop Constraint

(MySQL syntax may vary by version.)

```
ALTER TABLE Student
```

```
DROP CHECK chk_marks;
```

Complete Database Example

Department Table

```
CREATE TABLE Department
```

```
(
```

```
Dept_ID INT PRIMARY KEY,
```

```
Dept_Name VARCHAR(30)
```

```
);
```

Insert Data

```
INSERT INTO Department
```

```
VALUES
```

```
(1,'Computer'),
```

```
(2,'Mechanical'),
```

```
(3,'Civil');
```

Student Table

```
CREATE TABLE Student
```

```
(
```

```
Student_ID INT PRIMARY KEY,
```

```
Name VARCHAR(50),
```

```
Age INT,
```

```
Gender CHAR(1),
```

```
City VARCHAR(30),
```

```
Marks DECIMAL(5,2),
```

```
Dept_ID INT,
```

```
FOREIGN KEY (Dept_ID)
```

```
REFERENCES Department(Dept_ID)
```

```
);
```

Insert Data

INSERT INTO Student

VALUES

(101,'Rahul',20,'M','Delhi',85,1),
(102,'Aman',22,'M','Mumbai',91,1),
(103,'Priya',19,'F','Delhi',78,2),
(104,'Sneha',21,'F','Indore',88,2),
(105,'Rohan',20,'M','Pune',65,3),
(106,'Anjali',22,'F','Delhi',92,1),
(107,'Riya',23,'F','Mumbai',81,3),
(108,'Vikas',20,'M','Jaipur',75,2);

4. Data Manipulation Language (DML)

DML commands manipulate the data inside tables.

Commands:

- INSERT
 - UPDATE
 - DELETE
 - SELECT
-

INSERT Command

Adds records.

Syntax

INSERT INTO table_name

VALUES (...);

Example

INSERT INTO Student

VALUES

(109,'Karan',21,'M','Bhopal',82,1);

UPDATE Command

Changes existing records.

Syntax

UPDATE table_name

SET column=value

WHERE condition;

Example

Rahul's marks changed to 95.

UPDATE Student

SET Marks=95

WHERE Student_ID=101;

Before

Rahul = 85

After

Rahul = 95

Multiple columns

UPDATE Student

SET City='Jaipur',

Marks=88

WHERE Student_ID=103;

DELETE Command

Deletes rows.

DELETE FROM Student

WHERE Student_ID=108;

Only one record removed.

Delete all rows

DELETE FROM Student;

Table remains.

TRUNCATE

Deletes all rows quickly.

TRUNCATE TABLE Student;

Table becomes empty.

DROP TABLE

Deletes table completely.

DROP TABLE Student;

SELECT Statement

Used to retrieve data.

SELECT * FROM Student;

Output

|ID|Name|Marks|

All rows displayed.

Select particular columns

SELECT Name,Marks

FROM Student;

WHERE Clause

Filters records.

```
SELECT *
```

```
FROM Student
```

```
WHERE City='Delhi';
```

Output

Only Delhi students.

Another Example

```
SELECT Name,Marks
```

```
FROM Student
```

```
WHERE Marks>80;
```

Output

Rahul

Aman

Sneha

Anjali

Karan

Comparison Operators

Equal

```
WHERE Marks=85
```

Greater

```
WHERE Marks>80
```

Less

```
WHERE Marks<70
```

Not Equal

```
WHERE Marks<>80
```

DISTINCT

Removes duplicate values.

Without DISTINCT

SELECT City

FROM Student;

Output

Delhi

Delhi

Mumbai

Delhi

Pune

Mumbai

Jaipur

With DISTINCT

SELECT DISTINCT City

FROM Student;

Output

Delhi

Mumbai

Pune

Jaipur

Indore

ORDER BY

Sorts records.

Ascending

SELECT *

FROM Student

ORDER BY Marks;

Descending

SELECT *

FROM Student

ORDER BY Marks DESC;

Multiple columns

SELECT *

FROM Student

ORDER BY City,Marks DESC;

First City sorted.

Within City Marks sorted.

GROUP BY

Groups same values.

Average Marks Department-wise

SELECT Dept_ID,

AVG(Marks)

FROM Student

GROUP BY Dept_ID;

Output

|Dept_ID|AVG|

1

2

3

Count students

```
SELECT City,  
COUNT(*)  
FROM Student  
GROUP BY City;
```

HAVING Clause

Filters groups.

Difference

WHERE

↓

Filters rows

HAVING

↓

Filters groups

Example

Department having average marks above 80

```
SELECT Dept_ID,
```

```
AVG(Marks)
```

```
FROM Student
```

```
GROUP BY Dept_ID
```

```
HAVING AVG(Marks)>80;
```

WHERE + GROUP BY + HAVING

Example

Find department average marks above 80 considering only female students.

```
SELECT Dept_ID,
```

```
AVG(Marks)
```

FROM Student
WHERE Gender='F'
GROUP BY Dept_ID
HAVING AVG(Marks)>80;

Execution Order

Student Table

↓

WHERE

↓

GROUP BY

↓

HAVING

↓

SELECT

Special Operators

1. IN Operator

Checks multiple values.

Instead of

```
WHERE City='Delhi'  
OR City='Mumbai'  
OR City='Jaipur'  
Use  
SELECT *  
FROM Student  
WHERE City IN  
('Delhi','Mumbai','Jaipur');
```

2. BETWEEN

Checks range.

```
SELECT *  
FROM Student  
WHERE Marks  
BETWEEN 70 AND 90;  
Includes 70 and 90.
```

3. LIKE

Pattern matching.

Starts with A

```
SELECT *  
FROM Student  
WHERE Name LIKE 'A%';
```

Ends with a

```
LIKE '%a'
```

Contains "an"

```
LIKE '%an%'
```

Second letter is a

LIKE '_a%'

Wildcards

%

Any number of characters

—

One character

4. EXISTS

Checks whether a subquery returns any rows.

Example: Show departments that have at least one student.

```
SELECT Dept_Name
FROM Department d
WHERE EXISTS (
    SELECT *
    FROM Student s
    WHERE s.Dept_ID = d.Dept_ID
);
```

How it works:

- For each department, SQL checks whether at least one matching student exists.
 - If yes, that department is displayed.
-

5. ANY Operator

Compares a value with **any one** value returned by a subquery.

Example: Find students whose marks are greater than **at least one** student in Department 2.

```
SELECT Name, Marks
FROM Student
WHERE Marks > ANY (
    SELECT Marks
    FROM Student
    WHERE Dept_ID = 2
);
```

Suppose Department 2 marks are: **78, 88, 75**

The condition becomes:

Marks > ANY (78, 88, 75)

A student qualifies if their marks are greater than **at least one** of these values (e.g., greater than 75).

6. ALL Operator

Compares a value with **every** value returned by a subquery.

Example: Find students whose marks are greater than **all** students in Department 2.

```
SELECT Name, Marks
FROM Student
WHERE Marks > ALL (
    SELECT Marks
    FROM Student
    WHERE Dept_ID = 2
);
```

If Department 2 marks are:

78, 88, 75

The condition becomes:

Marks > 78

AND Marks > 88

AND Marks > 75

So only students with marks greater than **88** are returned.

Complete Query Flow (How Commands Relate)

Step 1: Create Tables

|



CREATE TABLE Department (...);

CREATE TABLE Student (...);

|



Step 2: Insert Data

|



INSERT INTO Department ...;

INSERT INTO Student ...;

|



Step 3: Retrieve Data

|



SELECT * FROM Student;

|



Step 4: Filter Rows

|



```
SELECT * FROM Student
```

```
WHERE Marks > 80;
```

|



Step 5: Remove Duplicates

|



```
SELECT DISTINCT City
```

```
FROM Student;
```

|



Step 6: Sort Results

|



```
SELECT * FROM Student
```

```
ORDER BY Marks DESC;
```

|



Step 7: Group Data

|



```
SELECT Dept_ID, AVG(Marks)
FROM Student
GROUP BY Dept_ID;
```



Step 8: Filter Groups



```
SELECT Dept_ID, AVG(Marks)
FROM Student
GROUP BY Dept_ID
HAVING AVG(Marks) > 80;
```



Step 9: Use Special Operators



IN, BETWEEN, LIKE, EXISTS, ANY, ALL



Step 10: Modify Data



```
UPDATE Student
```

SET Marks = 95

WHERE Student_ID = 101;

|



Step 11: Delete Data

|



DELETE FROM Student

WHERE Student_ID = 108;



MANAGING TABLES AND DATA (MySQL)

Creating & Altering Tables | Data Manipulation | Retrieval using SELECT | Special Operators

1. CREATING TABLES (DDL)

A table is a collection of related data in rows and columns.

Syntax:

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints,  
    ...  
);
```

Example:

Department Table

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(50) NOT NULL  
);
```

Student Table

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT,  
    Gender CHAR(1),  
    City VARCHAR(30),  
    Marks DECIMAL(5,2),  
    Dept_ID INT,  
    FOREIGN KEY (Dept_ID)  
    REFERENCES Department(Dept_ID)  
);
```

2. CONSTRAINTS

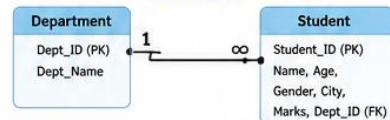
Constraints maintain data accuracy and integrity.

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each record. Cannot be NULL.	Student_ID INT PRIMARY KEY
NOT NULL	Column cannot contain NULL.	Name VARCHAR(50) NOT NULL
UNIQUE	No duplicate values allowed.	Email VARCHAR(100) UNIQUE
CHECK	Checks condition before inserting.	Age INT CHECK (Age >= 18)
DEFAULT	Automatically inserts value.	City VARCHAR(30) DEFAULT 'Bhopal'
FOREIGN KEY	Maintains relationship between tables.	Dept_ID INT FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)

3. ALTER TABLE (Modify Structure)

Add Column	ALTER TABLE Student ADD Phone VARCHAR(15);
Modify Column	ALTER TABLE Student MODIFY Phone VARCHAR(20);
Rename Column	ALTER TABLE Student RENAME COLUMN Phone TO Mobile;
Drop Column	ALTER TABLE Student DROP COLUMN Mobile;
Add Constraint	ALTER TABLE Student ADD CONSTRAINT chk_marks CHECK (Marks >= 0);
Drop Constraint	ALTER TABLE Student DROP CONSTRAINT chk_marks;

RELATIONSHIP



COMPLETE DATABASE EXAMPLE

Department Table

```
CREATE TABLE Department (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(30)  
);  
  
INSERT INTO Department VALUES  
(1, 'Computer'),  
(2, 'Mechanical'),  
(3, 'Civil');
```

Student Table

```
CREATE TABLE Student (  
    Student_ID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT, Gender CHAR(1),  
    City VARCHAR(30),  
    Marks DECIMAL(5,2),  
    Dept_ID INT,  
    FOREIGN KEY (Dept_ID)  
    REFERENCES Department(Dept_ID)  
);
```

```
INSERT INTO Student VALUES  
(101, 'Rahul', 20, 'M', 'Delhi', 85.00, 1),  
(102, 'Aman', 22, 'M', 'Mumbai', 91.00, 1),  
(103, 'Priya', 19, 'F', 'Delhi', 78.00, 2),  
(104, 'Sneha', 21, 'F', 'Indore', 88.00, 2),  
(105, 'Rohan', 20, 'M', 'Pune', 65.00, 3),  
(106, 'Anjali', 22, 'F', 'Delhi', 92.00, 1),  
(107, 'Riya', 23, 'F', 'Mumbai', 81.00, 3),  
(108, 'Vikas', 20, 'M', 'Jaipur', 75.00, 2),  
(109, 'Karan', 21, 'M', 'Bhopal', 82.00, 1);
```

SAMPLE DATA (Student Table)

Student_ID	Name	Age	Gender	City	Marks	Dept_ID
101	Rahul	20	M	Delhi	85.00	1
102	Aman	22	M	Mumbai	91.00	1
103	Priya	19	F	Delhi	78.00	2
104	Sneha	21	F	Indore	88.00	2
105	Rohan	20	M	Pune	65.00	3
106	Anjali	22	F	Delhi	92.00	1
107	Riya	23	F	Mumbai	81.00	3
108	Vikas	20	M	Jaipur	75.00	2
109	Karan	21	M	Bhopal	82.00	1

4. DATA MANIPULATION (DML)

INSERT - Adds new records

```
INSERT INTO Student VALUES  
(109, 'Karan', 21, 'M', 'Bhopal', 82.1);
```

UPDATE - Modifies existing records

```
UPDATE Student  
SET Marks = 95  
WHERE Student_ID = 101;
```

```
UPDATE Student  
SET City = 'Jaipur', Marks = 88  
WHERE Student_ID = 103;
```

DELETE - Deletes specific records

```
DELETE FROM Student  
WHERE Student_ID = 108;
```

TRUNCATE - Deletes all rows

```
TRUNCATE TABLE Student;
```

DROP TABLE - Deletes table completely

```
DROP TABLE Student;
```

5. RETRIEVING DATA - SELECT STATEMENT

Basic SELECT

```
SELECT * FROM Student;
```

Select Specific Columns

```
SELECT Name, Marks FROM Student;
```

WHERE Clause - Filter rows

```
SELECT * FROM Student  
WHERE City = 'Delhi';
```

```
SELECT Name, Marks FROM Student  
WHERE Marks > 80;
```

DISTINCT - Remove duplicates

```
SELECT DISTINCT City FROM Student;
```

ORDER BY - Sort results

```
-- Ascending  
SELECT * FROM Student ORDER BY Marks;  
  
-- Descending  
SELECT * FROM Student ORDER BY Marks DESC;  
  
-- Multiple columns  
SELECT * FROM Student  
ORDER BY City, Marks DESC;
```

GROUP BY - Group rows

```
-- Average Marks Department-wise  
SELECT Dept_ID, AVG(Marks) AS Avg_Marks  
FROM Student  
GROUP BY Dept_ID;  
  
-- Count Students in each city  
SELECT City, COUNT(*) AS Total_Students  
FROM Student  
GROUP BY City;
```

HAVING - Filter groups

```
-- Departments having average marks > 80  
SELECT Dept_ID, AVG(Marks) AS Avg_Marks  
FROM Student  
GROUP BY Dept_ID  
HAVING AVG(Marks) > 80;
```

WHERE + GROUP BY + HAVING

```
-- Average marks of female students > 80  
SELECT Dept_ID, AVG(Marks) AS Avg_Marks  
FROM Student  
WHERE Gender = 'F'  
GROUP BY Dept_ID  
HAVING AVG(Marks) > 80;
```

Execution Order

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

6. SPECIAL OPERATORS

IN Operator

Checks multiple values.

```
SELECT * FROM Student  
WHERE City IN  
( 'Delhi', 'Mumbai', 'Jaipur' );
```

(Instead of multiple OR)



BETWEEN Operator

Checks range (inclusive).

```
SELECT * FROM Student  
WHERE Marks  
BETWEEN 70 AND 90;
```



LIKE Operator

Pattern matching (uses % and _).

```
SELECT * FROM Student  
WHERE Name LIKE 'A%'; -- Starts with A  
WHERE Name LIKE '%a'; -- Ends with a  
WHERE Name LIKE '%an%'; -- Contains 'an'  
WHERE Name LIKE '_a%'; -- 2nd letter is a
```

Wildcard	Meaning
%	Any number of characters
_	Exactly one character

EXISTS Operator

Checks if subquery returns any rows.

```
SELECT Dept_Name  
FROM Department d  
WHERE EXISTS (  
    SELECT * FROM Student s  
    WHERE s.Dept_ID = d.Dept_ID  
);
```

Returns departments that have at least one student.

ANY Operator

Compare with ANY value from subquery.

```
SELECT Name, Marks  
FROM Student  
WHERE Marks > ANY (  
    SELECT Marks  
    FROM Student  
    WHERE Dept_ID = 2  
);
```

Greater than at least one value.

ALL Operator

Compare with ALL values from subquery.

```
SELECT Name, Marks  
FROM Student  
WHERE Marks > ALL (  
    SELECT Marks  
    FROM Student  
    WHERE Dept_ID = 2  
);
```

Greater than every value.

HOW QUERIES ARE RELATED - FLOW



★ Practice these queries in MySQL to master database management and data handling! 💡

Summary Table

SQL Feature	Purpose	Example
CREATE TABLE	Create a new table	CREATE TABLE Student (...);
ALTER TABLE	Modify table structure	ALTER TABLE Student ADD Phone VARCHAR(15);
Constraints	Enforce data integrity	PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK, UNIQUE, DEFAULT
INSERT	Add new records	INSERT INTO Student VALUES (...);
UPDATE	Modify existing records	UPDATE Student SET Marks=95 WHERE Student_ID=101;
DELETE	Remove specific records	DELETE FROM Student WHERE Student_ID=108;
SELECT	Retrieve data	SELECT * FROM Student;
WHERE	Filter rows	WHERE Marks > 80
DISTINCT	Remove duplicate values	SELECT DISTINCT City FROM Student;
ORDER BY	Sort results	ORDER BY Marks DESC
GROUP BY	Group rows for aggregation	GROUP BY Dept_ID
HAVING	Filter grouped results	HAVING AVG(Marks) > 80
IN	Match multiple values	WHERE City IN ('Delhi','Mumbai')
BETWEEN	Select a range	WHERE Marks BETWEEN 70 AND 90
LIKE	Pattern matching	WHERE Name LIKE 'A%'
EXISTS	Check if a subquery returns rows	WHERE EXISTS (...)
ANY	Compare with at least one value from a subquery	Marks > ANY (...)

SQL Feature	Purpose	Example
ALL	Compare with every value from a subquery	Marks > ALL (...)